NEPAL ENGINEERING COLLEGE

(Affiliated To Pokhara University)

Changunarayan, Bhaktapur

Report-03 on

Finite Automata and Pattern Recognition

Submitted by:
Name: Dharmdev Chai
Roll No:023-348
Date: 2nd june 2026

Submitted to:
Department of Computer Science and Engineering

Title: Finite Automata and Pattern Recognition

Objective

- To design and implement separate C programs using simple conditional logic (if-else statements and loops) to simulate Finite Automata for recognizing strings matching specific regular expression patterns.

Theory

A **Finite Automaton** transitions through states based on input characters to determine if a string matches a specific pattern (Regular Expression).

- An **Accepted** string drives the logic to a valid final state at the end of the input string.
- A **Rejected** string encounters unexpected characters or fails to reach the destination criteria.

Programs and Outputs

i. **Pattern: a* (Zero or more 'a's)**

```
#include <stdio.h>
int main() {
   char str[100];
   int i = 0, isValid = 1;
   printf("Enter a string for a*: ");
   scanf("%s", str);

   while (str[i] != '\0') {
      if (str[i] != 'a') {
         isValid = 0;
         break;
      }
      i++;
   }

   if (isValid == 1) {
      printf("Accepted\n");
   } else {
      printf("Rejected\n");
   }
   return 0;
}
```

Output:

```
┌──(dharmdev㉿kali)-[~/…/6thSem/compiler_design/lab_3_google_classroom/i]
└─$ ./main
Enter a string for a*: aaaa
Accepted
```

ii. Pattern: a+b (One or more 'a's followed by exactly one 'b')

```c
#include <stdio.h>

int main() {
    char str[100];
    int i = 0;

    printf("Enter a string for a+b: ");
    scanf("%s", str);

    if (str[0] != 'a') {
        printf("Rejected\n");
        return 0;
    }

    while (str[i] == 'a') {
        i++;
    }
    if (str[i] == 'b' && str[i+1] == '\0') {
        printf("Accepted\n");
    } else {
        printf("Rejected\n");
    }
    return 0;
}
```

Output:

```
┌──(dharmdev㉿kali)-[~/…/6thSem/compiler_design/lab_3_google_classroom/ii]
└─$ ./main
Enter a string for a+b: ab
Accepted
```

iii. Pattern: ab (Exactly 'ab')

```c
#include <stdio.h>

int main() {
    char str[100];

    printf("Enter a string for ab: ");
    scanf("%s", str);

    if (str[0] == 'a' && str[1] == 'b' && str[2] == '\0') {
        printf("Accepted\n");
    } else {
        printf("Rejected\n");
    }
    return 0;
```

```
}
```

Output:

```
┌──(dharmdev㉿kali)-[~/…/6thSem/compiler_design/lab_3_google_classroom/iii]
└─$ ./main
Enter a string for ab: ab
Accepted
```

iv. Pattern: abba (Exactly 'abba')

```
#include <stdio.h>

int main() {
    char str[100];

    printf("Enter a string for abba: ");
    scanf("%s", str);

    if (str[0] == 'a' && str[1] == 'b' && str[2] == 'b' && str[3] == 'a' && str[4] == '\0') {
        printf("Accepted\n");
    } else {
        printf("Rejected\n");
    }
    return 0;
}
```

Output:

```
┌──(dharmdev㉿kali)-[~/…/6thSem/compiler_design/lab_3_google_classroom/iv]
└─$ ./main
Enter a string for abba: abbaa
Rejected
```

v. Pattern: (a+b)* (Any combination of 'a' and 'b')

```
#include <stdio.h>

int main() {
    char str[100];
    int i = 0, isValid = 1;

    printf("Enter a string for (a+b)*: ");
    scanf("%s", str);

    while (str[i] != '\0') {
        if (str[i] != 'a' && str[i] != 'b') {
            isValid = 0;
            break;
        }
        i++;
    }
```

```
    if (isValid == 1) {
        printf("Accepted\n");
    } else {
        printf("Rejected\n");
    }
    return 0;
}
```

Output:

```
┌──(dharmdev㉿kali)-[~/…/6thSem/compiler_design/lab_3_google_classroom/v]
└─$ ./main
Enter a string for (a+b)*: abbaab
Accepted
```

vi. Pattern: a*b* (Any number of 'a's followed by any number of 'b's)

```
#include <stdio.h>

int main() { char str[100]; int i = 0;

printf("Enter a string for a*b*: ");
scanf("%s", str);
while (str[i] == 'a') {
    i++;
}
while (str[i] == 'b') {
    i++;
}
if (str[i] == '\0') {
    printf("Accepted\n");
} else {
    printf("Rejected\n");
}
return 0;

}
```

Output:

```
┌──(dharmdev㉿kali)-[~/…/6thSem/compiler_design/lab_3_google_classroom/vi]
└─$ ./main
Enter a string for a*b*: aababa
Rejected
```

Conclusion

By breaking down each regular expression into separate programs, the execution behavior of individual finite automata was successfully simulated. Using elementary if conditions and while loops provides a direct, highly readable way to enforce token evaluation rules without complex framework overhead.